

Struktúrák Pythonban: a @dataclass

Jegyzet és feladatlap

Hogyan dolgozz ezzel a jegyzettel?

Ugyanúgy, mint a függvényes jegyzettel: egyedül is végig tudsz menni rajta.

- **Kis lépések.** Minden újdonságot előbb egy kész kódon próbálsz ki, és csak utána írsz sajátot.
- **Ellenőrzés.** A feladatok megadják, mit kell kapnod. Ha a programod ugyanazt írja ki, mehetsz tovább.
- **Hibakereső.** Ha hibaüzenetet kapsz, lapozz a jegyzet végére, és keresd meg a táblázatban.

A jelölések a szokásosak: **Próbáld ki**, **Tippelj**, **Feladat**. A nehézséget csillagok jelzik: ★ = alap, ★★ = közepes, ★★★ = nehezebb.

Ha elakadsz: olvasd el újra a feladat előtti mintakódot, nézd meg a Hibakeresőt, kérdezd meg a padtársadat, és csak ezután szólj a tanárodnak. A C++-os társaid ugyanezt a témát **struct** néven tanulják, ugyanezekkel a feladatokkal.

1. Összetartozó adatok egy egységben

A **@dataclass** segítségével saját típust készíthetsz, amely több összetartozó adatot fog össze egyetlen változóban.

Próbáld ki - adatok szétszórva

Három diák nevét, életkorát és átlagát tároljuk. Eddigi tudásunkkal három külön lista kell hozzá:

```
nevek = ["Anna", "Bence", "Csilla"]
korok = [17, 16, 18]
atlagok = [4.5, 3.8, 4.9]

for i in range(len(nevek)):
    print(nevek[i], korok[i], atlagok[i])
```

Futtasd, aztán vegyél fel egy negyedik diákot: Dani, 17 éves, átlaga 2.9. Hány sort kellett módosítanod? Mi történik, ha az egyik listából kifelejtetted az új adatot?

A gond: Anna adatai három különböző helyen vannak. Semmi nem mutatja, hogy összetartoznak.

Próbáld ki - az első saját típus

```
from dataclasses import dataclass

@dataclass
class Diak:
    nev: str
    kor: int
    atlag: float

d = Diak("Anna", 17, 4.5)

print(d.nev, d.kor, d.atlag)
```

Nézzük meg a részeit:

- `from dataclasses import dataclass` – ez a sor a program legelejére kerül. Enélkül a Python nem ismeri a `@dataclass` jelölést.
- `@dataclass` – azt jelzi, hogy a következő sorokban egy adatokat összefogó típust írunk le.
- `class Diak:` – az új típus neve `Diak`. A saját típusok nevét nagy kezdőbetűvel szokás írni.
- A behúzott sorok a **mezők**: a mező neve, kettőspont, és az, hogy milyen adat kerül bele (`str` szöveg, `int` egész, `float` tizedes tört).
- `Diak("Anna", 17, 4.5)` – létrehozunk egy diákot. Az értékek abban a sorrendben kerülnek a mezőkbe, ahogy a mezők a leírásban állnak.
- `d.nev` – a `d` változó `nev` mezője. A mezőket ponttal éred el.

A típus leírása olyan, mint egy üres űrlap terve: megmondja, milyen rovatok vannak rajta. A `d` változó egy kitöltött űrlap.

Kísérlet – az egész egyben

```
print(d)
```

Mit látsz? Hibakeresésnél ez nagyon hasznos: egyetlen sorral megnézheted az összes mező értékét.

Kísérlet – hiányzó adatok

Próbáld ki ezt a két sort egyesével. Olvasd el a hibaüzenet utolsó sorát, és keresd meg a Hibakeresőben.

```
d = Diak()
d = Diak("Anna", 17)
```

Figyelem! A típus leírását a program elejére írd, a függvények és a főprogram elé. Létrehozáskor minden mezőnek értéket kell adnod.

Feladatok

1.1. feladat ★☆☆ – Saját adatok

Gépeld be az első saját típusos programot, és írd át benne az adatokat a sajátjaidra.

Ellenőrzés: egy sor jelenik meg a neveddel, az életkoroddal és egy átlaggal.

1.2. feladat ★☆☆ – Új mező

Vegyél fel a `Diak` típusba egy negyedik mezőt: `osztaly: str`. Adj neki értéket a létrehozáskor, és írd ki a többi adattal együtt.

Ellenőrzés: például `Anna 17 4.5 11.B`

1.3. feladat ★☆☆ – Két diák

Hozz létre két `Diak` típusú változót (`d1` és `d2`) különböző adatokkal, és írd ki őket két sorba. Hányszor kellett leírnod a típus leírását?

Ellenőrzés: két sor, két különböző diák adataival.

1.4. feladat ★☆☆ – Könyv

Írj `Konyv` nevű típust három mezővel: `cim` és `szerzo` (szöveg), `oldalszam` (egész). Hozz létre egy könyvet, és írd ki az adatait.

Ellenőrzés: például `Egri csillagok (Gardonyi Geza), 560 oldal`

2. Létrehozás, módosítás, másolás

A mezők ugyanúgy viselkednek, mint a közönséges változók: értéket kaphatnak, számolni lehet velük, beolvasott adat kerülhet beléjük.

Tippelj - számít a sorrend?

```
from dataclasses import dataclass

@dataclass
class Datum:
    ev: int
    honap: int
    nap: int

a = Datum(2026, 10, 4)
b = Datum(4, 10, 2026)

print(a.ev, b.ev)
```

A Python nem tudja, mit jelentenek a számaid: a helyes sorrendre neked kell figyelned. Ha biztosra akarsz menni, kiírhatod a mezők nevét is:

```
c = Datum(ev=2026, honap=10, nap=4)
```

Próbáld ki - módosítás és beolvasás

```
d = Diak("Anna", 17, 4.5)

d.kor = d.kor + 1
d.atlag = 4.7
print(d)

nev = input()
kor = int(input())
atlag = float(input())
uj = Diak(nev, kor, atlag)
print(uj)
```

Kísérlet - a Python nem ellenőrzi a típust

```
d = Diak("Anna", "tizenhet", 4.5)
print(d)
print(d.kor + 1)
```

A mezők mellé írt `int`, `str`, `float` csak tájékoztatás az olvasónak. A Python nem akadályozza meg, hogy más típusú adatot tegyél a mezőbe; a hiba csak később, a számolásnál derül ki.

Kísérlet - az elírt mezőnév

```
d = Diak("Anna", 17, 4.5)

d.atalg = 5.0
print(d.atlag)
print(d)
```

Hibát vártál? Megváltozott az átlag?

Figyelem! Ha értékadásnál elírod egy mező nevét, a Python nem jelez hibát, hanem szó nélkül létrehoz egy új mezőt ezzel a névvel. Ha egy mező „nem akar megváltozni”, először a nevét ellenőrizd!

Tippelj - másolat vagy ugyanaz?

```
d = Diak("Anna", 17, 4.5)
masik = d

masik.nev = "Bence"

print(d.nev, masik.nev)
```

Meglepő? Az `=` jel itt **nem** készít másolatot: a `masik` ugyanannak a diáknak egy második neve. Ugyanezt láttad a listáknál is a függvényes jegyzetben.

Ha valódi másolat kell, hozz létre egy új diákot ugyanazokkal az adatokkal:

```
masik = Diak(d.nev, d.kor, d.atlag)
```

Tippelj - egyformák-e?

```
a = Diak("Anna", 17, 4.5)
b = Diak("Anna", 17, 4.5)

print(a == b)
```

A `@dataclass` két változót akkor tekint egyenlőnek, ha minden mezőjük megegyezik.

Feladatok

2.1. feladat ★☆☆ - Dátum

Írd meg a `Datum` típust (`ev`, `honap`, `nap`). Hozd létre a mai dátumot, és írd ki pontokkal elválasztva. **Tipp:** `f-string`.

Ellenőrzés: például `2026.10.4.`

2.2. feladat ★☆☆ - Beolvasott diák

Olvasd be egy diák nevét, életkorát és átlagát, hozz létre belőlük egy `Diak` típusú változót, majd írd ki így: `Anna (17 éves) atlaga: 4.5`

2.3. feladat ★☆☆ - Téglalap

Írj `Teglalap` típust két egész mezővel: `szelesseg` és `magassag`. Hozd létre egy 5×3 -as téglalapot, és a főprogramban számold ki a mezőiből a területét és a kerületét.

Ellenőrzés: terület 15, kerület 16.

2.4. feladat ★☆☆ - Ki az idősebb?

Hozd létre két diákot: `Diak("Anna", 17, 4.5)` és `Diak("Csilla", 18, 4.9)`. Írd ki annak a nevét, aki idősebb, majd annak a nevét, akinek jobb az átlaga.

Ellenőrzés: mindkét esetben `Csilla`. Cseréld ki Anna életkorát 19-re: az első válasz `Anna` lesz.

2.5. feladat ★☆☆ - Időpont

Írj `Ido` típust (`ora`, `perc`). Hozd létre egy 7 óra 45 perces időpontot, és írd ki, hány perc telt el éjfél óta. Ezután növeld meg az időpontot 30 perccel úgy, hogy a perc 60 alatt maradjon.

Ellenőrzés: 465 perc. A növelés után az időpont 8 óra 15 perc.

3. Saját típus és függvény

Egy saját típusú változó ugyanúgy lehet függvény paramétere vagy visszatérési értéke, mint egy szám. Így egyetlen paraméterben utazik az összes összetartozó adat.

Próbáld ki - diák mint paraméter

```
from dataclasses import dataclass

@dataclass
class Diak:
    nev: str
    kor: int
    atlag: float

def kiir(d):
    print(f"{d.nev} ({d.kor} éves) atlaga: {d.atlag}")

def kituno(d):
    return d.atlag >= 4.5

a = Diak("Anna", 17, 4.5)
b = Diak("Bence", 16, 3.8)

kiir(a)
kiir(b)

if kituno(a):
    print(a.nev, "kituno.")
```

Gondold végig: saját típus nélkül a **kiir** függvénynek három paraméter kellett volna. És ha a diáknak tíz adata van?

Próbáld ki - függvény, amely diákot ad vissza

```
def beolvas():
    nev = input()
    kor = int(input())
    atlag = float(input())
    return Diak(nev, kor, atlag)
```

A főprogramban így használd: **uj = beolvas()**, majd **kiir(uj)**

Tippelj - megváltozik-e?

```
def szulinap(d):
    d.kor = d.kor + 1

a = Diak("Anna", 17, 4.5)
szulinap(a)
kiir(a)
```

Hány éves Anna a kiírásban? Emlékezz az előző fejezet másolós kísérletére: a függvény nem másolatot kap, hanem magát a diákot. Amit a mezőin módosít, az megmarad.

Feladatok

3.1. feladat ★☆☆ - Kiíró

Gépeld be a **kiir** függvényt, és írd ki vele három különböző diákot.

Ellenőrzés: három sor, például **Anna (17 éves) atlaga: 4.5**

3.2. feladat ★☆☆ - Terület és kerület

A 2.3. feladat `Teglalap` típusához írd két függvényt: `terulet(t)` és `kerulet(t)`.

Ellenőrzés: az 5×3 -as téglalapra `terulet` $\rightarrow 15$, `kerulet` $\rightarrow 16$.

3.3. feladat ★☆☆ - Nagykorú

Írd `nagykoru(d)` függvényt.

Ellenőrzés: `Diak("Csilla", 18, 4.9)` $\rightarrow$ True, `Diak("Anna", 17, 4.5)` $\rightarrow$ False.

3.4. feladat ★☆☆ - Idősebb

Írd `idosebb(a, b)` függvényt, amely a két diák közül az idősebbet adja vissza. Az eredményt a `kiir` függvénnyel írd ki.

Ellenőrzés: Anna (17) és Csilla (18) közül Csilla adatai jelennek meg.

3.5. feladat ★☆☆ - Percekben

A 2.5. feladat `Ido` típusához írd `percben(i)` függvényt, és egy másikat, amely visszafelé alakít: `idobol(perc)`.

Ellenőrzés: `percben(Ido(7, 45))` $\rightarrow 465$. Az `idobol(465)` eredménye `Ido(ora=7, perc=45)`.

3.6. feladat ★☆☆ - Korábbi dátum

Írd `korabbi(a, b)` függvényt, amely megmondja, hogy az `a` dátum korábbi-e a `b`-nél. **Tipp:** előbb az évet hasonlítsd össze; ha egyenlő, a hónapot; ha az is egyenlő, a napot.

Ellenőrzés: `Datum(2026, 10, 4)` és `Datum(2026, 12, 1)` $\rightarrow$ True; `Datum(2026, 10, 4)` és `Datum(2025, 12, 31)` $\rightarrow$ False; `Datum(2026, 10, 4)` és `Datum(2026, 10, 4)` $\rightarrow$ False.

4. Függvény a típuson belül

Azt a függvényt, amely egy saját típus adataival dolgozik, a típus leírásába is beírhatod. Ezt **metódusnak** hívjuk.

Próbáld ki

```
from dataclasses import dataclass

@dataclass
class Teglalap:
    szelesseg: int
    magassag: int

    def terulet(self):
        return self.szelesseg * self.magassag

t = Teglalap(5, 3)

print(t.terulet())
```

Hasonlítsd össze a 3.2. feladat megoldásával. Három dolog változott:

- A függvény a típus leírásába került, behúzva.
- A paramétere a `self`. Ez azt a téglalapot jelenti, amelyiken a metódust meghívtad. A mezőket a `self` segítségével éred el: `self.szelesseg`.
- A hívás ponttal történik, mint a mezők elérése: `t.terulet()`. A zárójelbe nem írsz semmit: a `self` helyére a Python a pont előtt álló változót teszi.

Ismerős ez a forma? A `szo.upper()` és a `lista.append(x)` is metódus volt.

Tippelj - kinek az adataival számol?

```
a = Teglalap(5, 3)
b = Teglalap(10, 10)

print(a.terulet(), b.terulet())
```

Próbáld ki - metódus paraméterrel, és amelyik módosít

Bővítsd a **Teglalap** típust ezzel a két metódussal. Figyelj a behúzásra!

```
def nagyobb(self, masik):
    return self.terulet() > masik.terulet()

def nagyit(self, szorzo):
    self.szelesseg = self.szelesseg * szorzo
    self.magassag = self.magassag * szorzo
```

A főprogramban próbáld ki:

```
a = Teglalap(5, 3)
b = Teglalap(4, 4)

print(a.nagyobb(b))

a.nagyit(2)
print(a)
print(a.terulet())
```

Figyeld meg:

- A **self** mindig az első paraméter. A többi paraméter utána következik, híváskor pedig csak azokat adod meg: **a.nagyobb(b)**.
- A metódus meghívhat egy másik metódust: **self.terulet()**.
- A **nagyit** megváltoztatja a mezőket, és ez a változás megmarad.

Kísérlet - a két leggyakoribb hiba

Rontsd el a **terulet** metódust kétféleképpen, **egyesével**, és keresd meg a hibaüzenetet a Hibakeresőben. Utána javítsd vissza.

Először hagyd el a **self** paramétert:

```
def terulet():
    return self.szelesseg * self.magassag
```

Aztán a mezők előtt hagyd el a **self**. előtagot:

```
def terulet(self):
    return szelesseg * magassag
```

Figyelem! A metódus első paramétere mindig a **self**, és a mezőkre a metóduson belül mindig **self**. előtaggal hivatkozol.

Tipp: Mikor írd a függvényt a típusba? Ha egyetlen változó adataiból számol vagy azokat módosítja (terület, kiírás, növelés), akkor legyen metódus. Mindkét megoldás helyes; válaszd azt, amelyikkel olvashatóbb a programod.

Feladatok

4.1. feladat ★☆☆ - Kerület

Írj a `Teglalap` típusba `kerulet` metódust.

Ellenőrzés: a `Teglalap(5, 3)` téglalapra `t.kerulet()` → 16.

4.2. feladat ★☆☆ - Négyzet-e?

Írj a `Teglalap` típusba `negyzet` metódust, amely megmondja, négyzet-e a téglalap.

Ellenőrzés: `Teglalap(4, 4)` → True, `Teglalap(5, 3)` → False.

4.3. feladat ★☆☆ - Kiíró metódus

Írd át a 3. fejezet `kiir` függvényét a `Diak` típus metódusává. A hívás ezután: `a.kiir()`

Ellenőrzés: Anna (17 éves) atlag: 4.5

4.4. feladat ★☆☆ - Kitűnő, nagykorú

Írj a `Diak` típusba két metódust: `kituno` (az átlag legalább 4.5) és `nagykoru`.

Ellenőrzés: `Diak("Csilla", 18, 4.9)` → mindkettő True; `Diak("Bence", 16, 3.8)` → mindkettő False.

4.5. feladat ★☆☆ - Időpont metódusokkal

Írd meg az `Ido` típust (`ora`, `perc`) két metódussal: a `percben` az éjfél óta eltelt percekot adja vissza, a `kiir` pedig kiírja az időpontot.

Ellenőrzés: az `Ido(7, 45)` időpontra `percben()` → 465, a `kiir()` kimenete 7:45.

4.6. feladat ★☆☆ - Bankszámla

Írj `Szaml` típust két mezővel: `tulajdonos` (szöveg) és `egyenleg` (egész). Metódusai: a `befizet(osszeg)` megnöveli az egyenleget; a `kivesz(osszeg)` csak akkor csökkenti, ha van elég pénz a számlán, és visszaadja, hogy sikerült-e.

Ellenőrzés: a számla 1000-ról indul. `befizet(500)` után az egyenleg 1500. `kivesz(2000)` → False, az egyenleg marad 1500. `kivesz(700)` → True, az egyenleg 800.

4.7. feladat ★★★ - Tört

Írj `Tort` típust (`szamlalo`, `nevezo`). Metódusai: az `ertek` a tört értékét adja vissza tizedes törtként; a `kiir` kiírja $\frac{3}{4}$ formában; az `egyszerusit` a lehető legegyszerűbb alakra hozza. **Tipp:** keresd meg a legnagyobb számot, amely a számlálót és a nevezőt is osztja.

Ellenőrzés: a `Tort(6, 8)` törtre `ertek()` → 0.75; `egyszerusit()` után a `kiir()` kimenete $\frac{3}{4}$.

5. Saját típusú elemek listája

Egy lista elemei saját típusú változók is lehetnek. Így sok diák, termék vagy versenyző összes adatát egyetlen listában tárolod – az érettségi feladatokban szinte mindig erre van szükség.

Próbáld ki

```
from dataclasses import dataclass

@dataclass
class Diak:
    nev: str
    kor: int
    atlag: float

diakok = [
    Diak("Anna", 17, 4.5),
    Diak("Bence", 16, 3.8),
    Diak("Csilla", 18, 4.9),
    Diak("Dani", 17, 2.9),
    Diak("Emma", 16, 4.2),
]

for d in diakok:
    print(d.nev, d.atlag)
```

A ciklusban a **d** sorban az egyes diákokat jelenti, a **d.nev** pedig az éppen soron következő diák nevét.

Hasonlítsd össze az 1. fejezet három listájával. Most hány sort kell írnod, ha új diák érkezik? És ha új mező kell?

Tippelj

Indexszel is elérheted az elemeket. Olvasd ki balról jobbra: **diakok[2].nev** – a lista 2-es indexű eleme (egy diák), annak pedig a **nev** mezője.

```
print(diakok[2].nev)
print(diakok[0].kor + diakok[1].kor)
print(diakok[-1].nev)
```

Próbáld ki - új elem hozzáadása

```
uj = Diak("Feri", 18, 3.5)
diakok.append(uj)

print(len(diakok))
```

Próbáld ki - a jól ismert minták, egy mezőre

A listás jegyzet minden mintája működik, csak most egy **mezőt** vizsgálsz. Megszámlálás: hány diák átlaga legalább 4.0?

```
db = 0
for d in diakok:
    if d.atlag >= 4.0:
        db = db + 1
print(db)
```

Maximumkiválasztás: ki a legidősebb? Magát a diákat jegyezzük meg, mert a végén a nevére vagyunk kíváncsiak, nem csak az életkorára.

```
legidosebb = diakok[0]
for d in diakok:
    if d.kor > legidosebb.kor:
        legidosebb = d
print(legidosebb.nev)
```

Feladatok

Mindegyik feladatban a fenti öt diák adataival dolgozz.

5.1. feladat ★☆☆ - Névsor

Írd ki minden diák összes adatát, soronként egy diákot. Ha megírtad a 4.3. feladat `kiir` metódusát, használd: `d.kiir()`

Ellenőrzés: öt sor, az első `Anna (17 éves) atlaga: 4.5`

5.2. feladat ★☆☆ - Az átlagok átlaga

Számold ki az öt diák átlagának átlagát, és írd ki két tizedesjegyre.

Ellenőrzés: 4.06.

5.3. feladat ★☆☆ - Tizenhét évesek

Hány diák 17 éves? Írd ki a nevüket is.

Ellenőrzés: 2 diák, Anna és Dani.

5.4. feladat ★☆☆ - A legjobb tanuló

Írd ki a legjobb átlagú diák nevét és átlagát.

Ellenőrzés: `Csilla 4.9`

5.5. feladat ★☆☆ - Keresés név alapján

Olvass be egy nevet, és írd ki az illető átlagát. Ha nincs ilyen nevű diák, ezt írd ki: `Nincs ilyen diak.`

Ellenőrzés: `Dani` → 2.9; `Zoli` → `Nincs ilyen diak.`

5.6. feladat ★☆☆ - Kiválogatás

Válogasd ki egy új listába azokat, akiknek az átlaga legalább 4.0. Írd ki az új lista hosszát és a neveket.

Ellenőrzés: 3 diák: Anna, Csilla, Emma.

5.7. feladat ★☆☆ - Beolvasás

Kérdezd meg, hány diák lesz. Olvasd be mindegyik nevét, életkorát és átlagát, és tedd őket egy listába. A végén írd ki a legfiatalabb diák nevét.

Ellenőrzés: ha a fenti öt diák adatait írod be, az eredmény `Bence`.

5.8. feladat ★★★ - Függvényekkel

Írj két függvényt: az `atlagok_atlaga(lista)` az átlagok átlagát, a `legjobb(lista)` a legjobb átlagú diákot adja vissza. A főprogram csak meghívja őket, és kiírja az eredményt.

Ellenőrzés: 4.06 (két tizedesjegyre) és Csilla adatai.

6. Egy sorból egy diák

A feladatokban az adatok soronként érkeznek, egy sorban egy diák (termék, versenyző) összes adata. Egy függvény a sorból saját típusú változót készít.

Próbáld ki

A sor feldarabolását a sztringes jegyzet 9. fejezetéből ismered. Most a darabokból egy **Diak** lesz:

```
from dataclasses import dataclass

@dataclass
class Diak:
    nev: str
    kor: int
    atlag: float

def feldolgoz(sor):
    reszek = sor.split(";")
    return Diak(reszek[0], int(reszek[1]), float(reszek[2]))

d = feldolgoz("Kiss Anna;17;4.5")

print(d.nev, "jovore", d.kor + 1, "eves lesz.")
```

Mi történne, ha az `int()` és a `float()` elmaradna? Próbáld ki, és nézd meg a `print(d)` kimenetét is.

Próbáld ki - sok sor, diákok listája

Cseréld le a főprogramot erre:

```
db = int(input())

diakok = []
for i in range(db):
    sor = input()
    diakok.append(feldolgoz(sor))

print(len(diakok), "diak adatai beolvasva.")
```

Ez a három lépés (sor beolvasása, átalakítása, listába tétele) szinte minden nagyobb feladat eleje. Ha ez megvan, a kérdésekre az 5. fejezet mintáival válaszolsz.

Feladatok

6.1. feladat ★☆☆ - Ellenőrző kiírás

Bővítsd a fenti programot: a beolvasás után írd ki minden diák nevét és átlagát.

Ellenőrzés: írd be ezt a három sort a darabszám (3) után, és nézd meg, hogy mind megjelenik-e:

```
Kiss Anna;17;4.5
Nagy Bence;16;3.8
Toth Csilla;18;4.9
```

6.2. feladat ★☆☆ - Termék

Írj **Termek** típust: `nev` (szöveg), `ar` és `darab` (egész). Legyen egy `ertek` metódusa, amely az ár és a darabszám szorzatát adja vissza. Írj hozzá `feldolgoz(sor)` függvényt a `nev;ar;darab` formájú sorokhoz.

Ellenőrzés: a `fuzet;350;12` sorból készült termékre `ertek()` → 4200.

6.3. feladat ★☆☆ - Raktár

Olvass be négy terméket egy listába, és válaszolj a kérdésekre: mennyi a teljes készlet értéke? Melyik a legdrágább termék? Melyik termék készlete ér a legtöbbet?

```
fuzet;350;12
toll;180;30
radir;120;8
vonalto;420;5
```

Ellenőrzés: a teljes érték 12660; a legdrágább a **vonalto**; a legtöbbet a **toll** készlete éri (5400).

6.4. feladat ★★★ - Szóközzel tagolt sor

Írj `feldolgoz2(sor)` függvényt olyan sorokhoz, amelyekben szóköz az elválasztó, és a név egyetlen szó: Anna 17 4.5.

Ellenőrzés: az Anna 17 4.5 sorból készült diák **kor** mezője 17, **atlag** mezője 4.5.

7. Rendezés egy mező szerint

A `sort()` nem tudja magától, melyik mező szerint rendezzen. Egy függvénnyel mondd meg neki.

Kísérlet - rendezés segítség nélkül

Az 5. fejezet programjában próbáld meg rendezni a diákokat:

```
diakok.sort()
```

Olvasd el a hibaüzenet utolsó sorát. A Python nem tudja, mit jelent az, hogy az egyik diák „kisebb”, mint a másik.

Próbáld ki - rendezés kulcs szerint

Írd ezt a függvényt a főprogram elé. Azt adja vissza, ami **szerint** rendezni szeretnél:

```
def atlag_szerint(d):
    return d.atlag
```

A rendezésnél add meg a függvény **nevét** (zárójelek nélkül) **key** néven:

```
diakok.sort(key=atlag_szerint)

for d in diakok:
    print(d.nev, d.atlag)
```

A `sort` minden diákra meghívja a függvényedet, és a kapott értékek szerint rendez. Csökkenő sorrendhez írd ezt: `diakok.sort(key=atlag_szerint, reverse=True)`

Feladatok

7.1. feladat ★☆☆ - Névsor

Rendezd az öt diákot név szerint ábécérendbe, és írd ki a neveket.

Ellenőrzés: Anna, Bence, Csilla, Dani, Emma. (Ez már eleve így volt; hogy lásd a rendezést, keverd össze a diákokat a lista megadásakor.)

7.2. feladat ★☆☆ - Rangsor

Írd ki a diákokat átlag szerint csökkenő sorrendben, sorszámozva.

Ellenőrzés: 1. Csilla 4.9, 2. Anna 4.5, 3. Emma 4.2, 4. Bence 3.8, 5. Dani 2.9

7.3. feladat ★★★ - Kor, azon belül név

Rendezd a diákokat életkor szerint növekvő sorrendbe; az egyidősek névsorban kövessék egymást.

Tipp: a kulcsfüggvény két értéket is visszaadhat: `return d.kor, d.nev`. Ilyenkor az első dönt, egyezésnél a második.

Ellenőrzés: Bence, Emma, Anna, Dani, Csilla.

8. Típusok egymásban, és egy nagy feladat

Egy mező lehet másik saját típus vagy lista is. Így bonyolultabb adatokat is egy egységbe tudsz szervezni.

Próbáld ki - saját típus a típusban

```
from dataclasses import dataclass

@dataclass
class Datum:
    ev: int
    honap: int
    nap: int

@dataclass
class Tanulo:
    nev: str
    szuletes: Datum

t = Tanulo("Anna", Datum(2009, 5, 17))

print(t.nev, "születési éve:", t.szuletes.ev)
```

Olvasd ki balról jobbra: `t.szuletes.ev` – a `t` tanuló `szuletes` mezője (egy `Datum`), annak pedig az `ev` mezője.

Próbáld ki - lista a típusban

```
from dataclasses import dataclass

@dataclass
class Naplo:
    nev: str
    jegyek: list

    def atlag(self):
        osszeg = 0
        for jegy in self.jegyek:
            osszeg = osszeg + jegy
        return osszeg / len(self.jegyek)

n = Naplo("Anna", [5, 3, 4, 5, 2])
print(n.nev, "atlaga:", n.atlag())

n.jegyek.append(5)
print(n.nev, "atlaga:", n.atlag())
```

Most már nem kell az átlagot külön mezőben tárolni: a típus a jegyekből bármikor kiszámolja.

Feladatok

8.1. feladat ★★☆☆ - Életkor

Írj a `Tanulo` típusba `eletkor(ev)` metódust, amely megadja, hány éves lesz a tanuló a megadott év végén.

Ellenőrzés: a 2009. május 17-én született tanulóra `t.eletkor(2026)` → 17.

8.2. feladat ★★☆☆ - Napló

Bővítsd a `Naplo` típust két metódussal: a `legjobb` a legjobb jegyet, a `hany_otos` az ötösök számát adja vissza.

Ellenőrzés: az `[5, 3, 4, 5, 2]` jegyekre `legjobb()` → 5, `hany_otos()` → 2.

8.3. feladat ★★★★★ - Osztálynapló

Hozz létre egy listát három `Naplo` elemmel: Anna jegyei 5, 3, 4, 5, 2; Bence jegyei 4, 4, 3; Csilla jegyei 5, 5, 4, 5. Írd ki mindenki nevét az átlagával (két tizedesjegyre), majd a legjobb átlagú tanuló nevét.

Ellenőrzés: Anna 3.80, Bence 3.67, Csilla 4.75; a legjobb Csilla.

8.4. feladat ★★★★★ - Futóverseny

Ebben a feladatban mindent felhasználj, amit tanultál. Egy futóverseny eredményei soronként érkeznek, `rajtszam;nev;perc;masodperc` formában:

```
12;Kiss Anna;24;35
7;Nagy Bence;22;58
31;Toth Csilla;23;10
4;Szabo Dani;26;2
18;Varga Emma;22;41
```

Írj `Futo` típust a négy mezővel és egy `ossz_masodperc` metódussal. Írj `feldolgoz(sor)` függvényt. Olvasd be az öt sort egy listába (előbb a darabszámot), aztán válaszolj:

- Ki nyerte a versenyt, és mennyi volt az ideje?
- Hány futó ért be 25 percen belül?
- Mennyi az átlagos idő másodpercben?
- Mi a végeredmény? Írd ki a futókat a beérkezés sorrendjében.

Ellenőrzés: a győztes Varga Emma (22 perc 41 másodperc). 25 percen belül 4 futó ért be. Az átlagos idő 1433.2 másodperc. A sorrend: Varga Emma, Nagy Bence, Toth Csilla, Kiss Anna, Szabo Dani.

Építsd lépésenként: előbb csak a típus és a `feldolgoz` függvény egyetlen sorral kipróbálva, aztán a beolvasás, és csak utána a kérdések, egyenként.

Hibakereső: mit jelent a hibaüzenet?

A Python hibaüzenetének mindig az **utolsó sorát** olvasd el először. Keresd meg a táblázatban azt a sort, amely hasonlít rá.

Amit látsz	Mit jelent?	Mit tegyél?
<code>NameError: name 'dataclass' is not defined</code>	Hiányzik a program elejéről az import sor.	Írd a legelső sorba: <code>from dataclasses import dataclass</code>
<code>NameError: name 'Diak' is not defined</code>	A Python nem ismeri ezt a típust.	A típus leírása a használata előtt van? Jól írtad a nevét (nagybetű)?
<code>TypeError: Diak.__init__() missing 1 required positional argument: 'atlag'</code>	Létrehozáskor kevesebb adatot adtál meg, mint ahány mező van.	Az üzenet megmondja, melyik mező maradt ki.
<code>TypeError: Diak.__init__() takes 4 positional arguments but 5 were given</code>	Létrehozáskor több adatot adtál meg, mint ahány mező van.	Számold meg a mezőket és a megadott értékeket.
<code>AttributeError: 'Diak' object has no attribute 'eletkor'</code>	Ilyen nevű mezője vagy metódusa nincs a típusnak.	Nézd meg a leírásban, pontosan hogy hívják.
<code>AttributeError: 'list' object has no attribute 'nev'</code>	A listának nincs ilyen mezője, csak egy-egy elemének.	Kimaradt az index vagy a ciklus: <code>diakok[i].nev</code>
<code>TypeError: Teglalap.terulet() takes 0 positional arguments but 1 was given</code>	A metódusból kimaradt a <code>self</code> paraméter.	Írd be elsőnek: <code>def terulet(self):</code>
<code>NameError: name 'szelesseg' is not defined (metódusban)</code>	A mező előtt kimaradt a <code>self.</code> előtag.	<code>self.szelesseg</code>
<code><bound method ...></code> jelenik meg a kiírásban	Lemaradt a zárójel a metódus hívásáról.	<code>t.terulet</code> helyett <code>t.terulet()</code>
<code>TypeError: '<' not supported between instances of 'Diak' and 'Diak'</code>	A rendezés nem tudja, melyik mező szerint rendezzen.	Adj meg kulcsfüggvényt: <code>sort(key=...)</code> (7. fejezet).
<code>TypeError: can only concatenate str</code> vagy furcsa eredmény a számolásnál	Egy mezőben szöveg van szám helyett.	A <code>feldolgoz</code> függvényben kell az <code>int()</code> , illetve a <code>float()</code> .
Nincs hiba, de a mező nem változott meg	Értékadásnál elírtad a mező nevét, és új mező jött létre.	Írasd ki: <code>print(d)</code> , és ellenőrizd a nevet.
Két változó egyszerre változik	A <code>masik = d</code> nem másolat, hanem ugyanaz a változó.	Hozz létre újat: <code>Diak(d.nev, d.kor, d.atlag)</code>

Összefoglaló táblázat (puska)

Minden program elejére kell: `from dataclasses import dataclass`

Minta	Jelentés
<code>@dataclass</code> és alatta <code>class Diak:</code>	új típus leírásának kezdete
<code>nev: str</code>	mező a típus leírásában (név és adatfajta)
<code>d = Diak("Anna", 17, 4.5)</code>	létrehozás, az értékek a mezők sorrendjében
<code>d.nev</code>	a <code>d</code> változó <code>nev</code> mezője
<code>print(d)</code>	az összes mező kiírása egyben
<code>a == b</code>	igaz, ha minden mező megegyezik
<code>def kiir(d):</code>	függvény saját típusú paraméterrel
<code>def terület(self):</code>	metódus a típuson belül; a mezőket <code>self.</code> előtaggal éri el
<code>t.terület()</code>	metódus hívása a <code>t</code> változó adataival
<code>diakok = [Diak(...), Diak(...)]</code>	saját típusú elemek listája
<code>for d in diakok:</code>	bejárás; a ciklusban <code>d.nev</code> , <code>d.kor</code>
<code>diakok[i].nev</code>	az <code>i</code> indexű elem <code>nev</code> mezője
<code>diakok.sort(key=fuggveny)</code>	rendezés a kulcsfüggvény által visszaadott érték szerint

Figyelem! Három dolog, amit soha ne felejts el: a metódus első paramétere a `self`, a mezőket benne `self.` előtaggal éred el; az elírt mezőnév értékadásnál nem ad hibát; a `masik = d` nem készít másolatot.